

# 车载平衡滚球运动控制系统（H 题）

## MaixCAM 方案：底盘控制与机械机构阶段初稿

**阶段范围：**本文只完成底盘循迹、球杆机械机构、位置 PD 到步进电机的控制链设计。MaixCAM 内部的小球识别算法、图像标定结果和实测数据由视觉组后续补充。本文所有文字均按本队方案重新组织，未沿用他人的视觉与执行机构方案表述。

**摘要：**系统采用底盘与滚球机构相对独立、统一供电和协同调试的结构。底盘以 MSPM0G3507 为控制器，利用 8 路数字红外阵列获得黑线横向偏差，经位置 PD 生成目标偏航角速度，再由陀螺仪偏航 PI 和左右轮编码器速度 PID 形成差速闭环，实现循迹、分段调速、计时和终点停车。滚球机构由 MaixCAM 直接取得小球位置，计算目标位置误差及其变化率，位置 PD 输出目标球杆角度；该角度经摇杆机构标定表换算为步进电机绝对角度，并通过 UART 发送位置指令。机械结构采用一端固定铰接、另一端由步进电机摇臂和连杆抬升的单自由度方案，使电机旋转转化为球杆端部上下位移。本文给出当前代码对应的控制链、接口与安全约束，并将尚未测量的结构尺寸、传动标定和控制参数明确列为后续实测项。

**关键词：**MaixCAM；8 路红外循迹；差速底盘；位置 PD；步进电机；摇杆机构

## 1 系统方案与任务分工

整车由循迹底盘、球位视觉、单自由度球杆、步进电机执行机构及电源与人机交互模块组成。底盘控制器负责车辆沿黑线运行、轮速闭环、里程与计时；MaixCAM 负责球位测量、滚球位置 PD 以及步进电机串口控制。这样可避免把高频底盘闭环依赖于视觉帧率，同时又保留 MaixCAM 直接驱动球杆机构的简洁链路。两部分共用启动状态和安全策略，但各自保持独立的实时任务。

底盘与球杆执行机构的分工关系

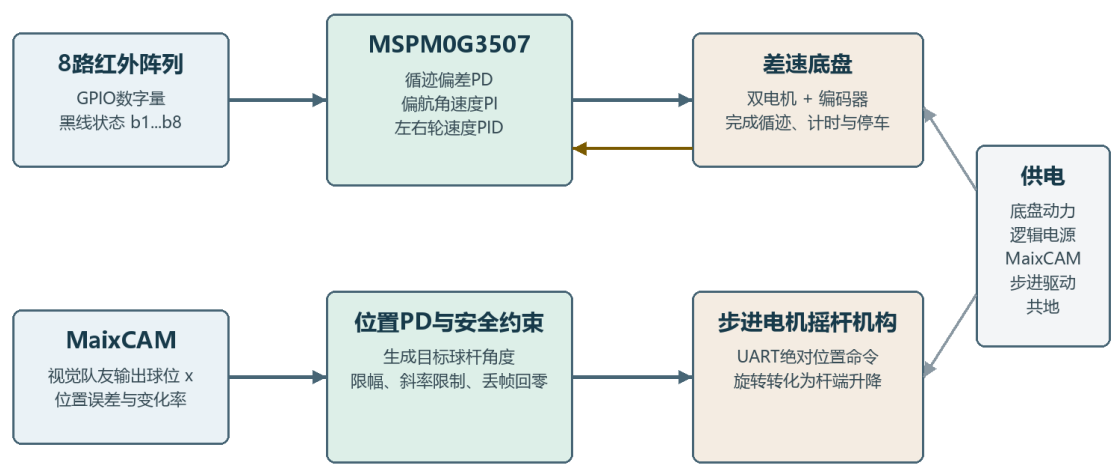


图 1 底盘与球杆执行机构的总体分工

| 模块   | 本阶段采用方案                           | 主要输入/输出                   |
|------|-----------------------------------|---------------------------|
| 循迹底盘 | MSPM0G3507 + 8 路数字红外 + IMU + 双编码器 | 黑线状态、角速度、轮速 -> 左右电机 PWM   |
| 球位视觉 | MaixCAM 直接测得小球沿杆方向的位置             | 输出 x、有效标志和采样时刻；识别细节待视觉组补充 |
| 滚球控制 | 位置 PD 直接生成目标球杆角度                  | 目标位置与球位 -> 球杆角度命令         |
| 机械执行 | 步进电机 + 摇臂/连杆 + 单端铰接球杆             | 电机绝对角 -> 杆端高度与球杆倾角        |

2 底盘循迹控制设计

2.1 8 路红外采集与横向偏差

8 路红外传感器横向安装在车体前部，当前工程把 OUT1 至 OUT8 依次接至 PA27、PA26、PA25、PA24、PB25、PB24、PB20 和 PA22。各通道以 GPIO 数字量读取，默认低电平表示检测到黑线。控制任务每 5 ms 读取一次原始电平，并将有效状态记为  $b_i$ （检测到黑线时  $b_i=1$ ，否则为 0）。

为把离散通道变为连续偏差，按从左到右给各传感器分配位置权重  $w_i$ 。当前可执行代码中的权重为{-3, -2, -1, -0.1, 0.1, 1, 2, 3}，比模板中常见的等间距权重更强调中间两路的小误差区。多路同时压线时取已触发通道权重的平均值，再以 3.5 归一化并限幅。

$$e_y = \text{sat}[-1,1]\{ [\Sigma(w_i b_i) / \Sigma b_i] / 3.5 \}$$

当 8 路原始状态全同，程序保持上一帧识别结果，避免瞬时强光、脱线或全黑横线使偏差突变；若在其他组合下没有有效通道，偏差函数返回 0。正式比赛前应在实际黑线宽度、安装高度和环境照度下重新核对有效电平、权重与脱线恢复方向，不能只依据注释确定参数。

## 2.2 位置、偏航与轮速三级闭环

底盘控制按“黑线横向误差 → 目标偏航角速度 → 左右轮目标速度 → 电机 PWM”逐级计算。最外层位置 PD 把归一化偏差  $e_y$  变为目标偏航角速度  $\omega_d$ 。比例项决定回线力度，微分项抑制高速循迹时的左右摆动；微分量经一阶低通处理，降低数字红外跳变带来的尖峰。

$$\omega_d(k) = \text{sat}\{K_p e_y(k) + K_d e [e_y(k) - e_y(k-1)] / T_s\}$$

偏航环使用 IMU 的 Z 轴角速度  $\omega_z$  作为反馈。启动前完成零偏校准，运行时由 PI 控制器根据  $\omega_d - \omega_z$  产生差速量  $\Delta v$ ，并进行积分与输出限幅。当前代码按下式合成左右轮目标速度，其中 0.4 为实车调试后的差速混合系数，并非由理想差速模型直接得到。

$$\Delta v = K_{p,\omega}(\omega_d - \omega_z) + K_{i,\omega} \int (\omega_d - \omega_z) dt$$

$$v_{L^*} = v_0 + 0.4\Delta v, \quad v_{R^*} = v_0 - 0.4\Delta v$$

左右轮分别使用编码器速度闭环。编码器每 5 ms 更新一次，当前工程以 3726 计数/m 换算轮速，再用新值权重 0.35 的一阶 IIR 滤波。增量式 PID 根据目标轮速与实测轮速计算 PWM，单次 P、I、D 增量和最终占空比均有限幅，避免启动或转弯时输出突变。最终 PWM 通过左右电机驱动接口输出。

$$\Delta u(k) = K_p [e(k) - e(k-1)] + K_i e(k) T_s + K_d [e(k) - 2e(k-1) + e(k-2)] / T_s$$

| 控制层          | 当前工程初值                                         | 说明                       |
|--------------|------------------------------------------------|--------------------------|
| 循迹位置 PD (K1) | $K_p=57.0$ , $K_d=2.1$ , $ \omega_d  \leq 500$ | 快车模式现有初值，必须结合实际传感器权重复调   |
| 偏航 PI        | $K_p=17.5$ , $K_i=15.0$ , 积分限幅 80              | 输入为目标/实测 Z 轴角速度          |
| 左右轮速度 PID    | $K_p=1.0$ , $K_i=80.0$ , $K_d=0$               | PWM 限幅 9000，单项增量限幅 1000  |
| 控制周期         | 5 ms (200 Hz)                                  | 红外、IMU、编码器和三层控制在同一控制帧内更新 |

**参数性质：**上表是当前工程中的起始参数，不是最终比赛参数。更换车重、轮胎、红外高度或球杆机构后都必须重新记录阶跃与循迹波形。

2.3 发车、调速与终点停车

现有工程设置 K1 和 K2 两种发车模式。K1 用于快速循迹和终点停车：启动时清零里程、使能三层闭环和横线检测；K2 用于低速联调：以较低初速起步并逐步提升到巡航速度。球杆安装完成后的首次联调应从低速模式开始，确认车体加速度不会使小球冲向端部，再逐步提高车速。

终点识别使用最近 20 个控制帧（约 100 ms）的滑动窗口。若窗口内 8 路中至少 4 路曾检测到低电平，程序认为传感器阵列扫过横向黑线，随即调用统一停车函数，清空 PID 历史、关闭控制使能、将目标速度置零并停止电机，同时锁存运行时间。该判据能适应小车斜着压过终点线，但必须增加起步屏蔽或里程门限，避免在起点附近和大曲率弯道误触发。

K1 当前在累计里程达到 2580 mm 后开始按步长降低基准速度。该阈值与场地一圈长度并不等价，只能视为当前样车的阶段参数。正式方案应根据完整场地长度、编码器实测比例和制动距离重新确定“允许识别终点线”的里程窗口，并通过连续多圈测试验证停车偏差。

2.4 底盘程序时序

| 执行顺序 | 函数/数据                          | 作用                  |
|------|--------------------------------|---------------------|
| 1    | MID_IR_ReadRaw / UpdateBits    | 读取 8 路 GPIO 并生成黑线状态 |
| 2    | MID_Line_CalcError             | 由位置权重计算归一化横向偏差      |
| 3    | MID_Chassis_PosStep            | 位置 PD 生成目标偏航角速度     |
| 4    | MID_Line_CheckEmergency        | 滑动窗口判断终点横线并触发停车     |
| 5    | MID_Chassis_YawStep            | IMU 偏航 PI 生成左右轮差速   |
| 6    | MID_Encoder_Update / SpeedStep | 编码器速度 PID 输出左右 PWM  |
| 7    | MID_Chassis_RampStep           | 分频执行发车加速或里程减速       |

3 球杆机械机构设计

3.1 结构组成与布置

球杆沿小车纵向中心线布置，一端通过支座铰接在车体上，构成固定转动中心；另一端作为驱动端，由步进电机输出轴上的摇臂带动连杆上下运动。摇臂转动后改变连杆连接点的高度，从而抬高或降低球杆驱动端，使球杆绕固定铰点产生小角度转动。该结构只有一个受控自由度，传力路径短，便于在 MaixCAM 中建立“目标球杆角度—电机绝对角度”的标定表。

## 单自由度球杆机械机构示意

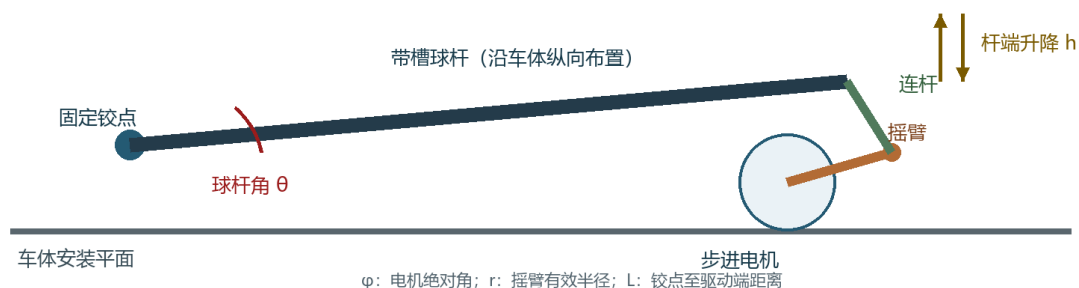


图 2 步进电机摇杆带动球杆单端升降的结构示意

固定铰点、驱动端连接点和步进电机轴线应安装在同一纵向工作平面内，减小连杆侧向扭曲。驱动端连接件需允许小范围摆动，避免机构运动时卡死；球杆在水平位置时应处于摇臂有效行程的中部，为正、负两个方向留出相近余量。摄像头支架与球杆支座需要有独立刚度，防止步进电机动作造成视场相对球杆移动。

### 3.2 运动学关系与标定

设步进电机相对水平零点的转角为  $\phi$ ，摇臂有效半径为  $r$ ，摇臂在水平零点处的安装相位为  $\phi_0$ ，固定铰点到驱动端的有效距离为  $L$ 。忽略连杆摆角和安装偏置时，驱动端相对升降量与球杆倾角可近似写为：

$$h(\phi) \approx r[\sin(\phi + \phi_0) - \sin\phi_0], \quad \theta(\phi) = \arctan[h(\phi)/L] \approx h(\phi)/L$$

实际机构还包含连杆长度、连接孔偏置和装配间隙， $\theta$  与  $\phi$  通常不是严格线性关系。因此控制程序不直接使用理论比例，而是在球杆上放置角度尺或电子水平仪，分别记录若干组“球杆实测角  $\theta$ —电机绝对角  $\phi$ ”，再采用分段线性插值反求电机目标角。当前程序中的  $\{-2^\circ \text{对应} -6^\circ, 0^\circ \text{对应} 0^\circ, +2^\circ \text{对应} +6^\circ\}$  只是占位标定，必须由实物测量替换。

### 3.3 强度、回差与安全限位

步进电机选型应同时校核静态保持转矩和动态加速转矩。设驱动端等效竖向载荷为  $F$ ，摇臂半径为  $r$ ，传动效率为  $\eta$ ，安全系数为  $S$ ，则电机所需转矩至少满足  $T_m \geq SFr/\eta$ 。 $F$  应包括球杆自重、钢球在最不利位置的重力分量、连杆惯性和车辆振动附加载荷。待杆长、质量、摇臂半径与电机型号确定后，再代入实测值完成报告中的数值计算。

机械端应设置硬限位，软件端同时设置球杆角度限幅、电机绝对角限幅和角度变化率限制。当前位置程序将正常平衡角限制在  $\pm 2^\circ$ ，电机机械保护范围限制在  $\pm 40^\circ$ ，命令死区为  $0.10^\circ$ ，角度变化

率为  $8^\circ/\text{s}$ ；这些是保守初值。调试时应先拆下钢球，以低速、小角度检查正负方向、零位和限位，再装球逐步增加控制幅度。

## 4 MaixCAM 位置 PD 与步进电机驱动

### 4.1 视觉模块输入边界

视觉算法由队友实现，本阶段只规定控制接口。MaixCAM 每次得到有效检测后，应向控制层提供小球沿球杆方向的位置  $x$ （单位  $\text{cm}$ ）、检测有效标志  $\text{valid}$  和采样时刻  $t_k$ 。像素到厘米的标定、反光抑制、异常点剔除和丢帧判断由视觉模块负责；控制层不得把无效坐标或过期坐标当作当前球位。

### 4.2 位置 PD 生成球杆角度

设目标球位为  $x_r$ ，位置误差定义为  $e = x_r - x$ 。位置 PD 直接根据当前误差与误差变化率生成目标球杆角度  $\theta_d$ 。由于目标位置在保持阶段不变，误差微分近似等于小球速度的相反数，因此 D 项为系统提供阻尼，可抑制小球越过目标点后的往复振荡。控制符号必须通过实物验证：若小球位于正方向，输出角度应使其产生朝负方向的加速度。

$$e(k) = x_r - x(k)$$

$$\dot{e}_f(k) = \text{LPF}\{[e(k) - e(k-1)]/T_k\}$$

$$\theta_d(k) = \text{sat}[-\theta_{\max}, \theta_{\max}]\{s[K_p e(k) + K_d \dot{e}_f(k)]\}$$

其中  $s$  为由机构正方向确定的符号（+1 或 -1）， $\theta_{\max}$  为允许球杆角度。为避免视觉噪声频繁驱动步进电机，控制输出还应经过小误差死区、角度斜率限制和命令死区；目标位置切换时宜使用斜坡或 S 形轨迹，避免直接阶跃。视觉短时丢帧可保持上一条命令，超过设定时间后应清除 PD 历史并缓慢回到水平角。

### 4.3 串口绝对位置控制

MaixCAM 通过 UART 直接连接步进电机驱动器，当前程序采用 `/dev/ttyS0`、115200 bit/s、电机地址 `0x01` 和校验字节 `0x6B`。上电后先读取当前位置确认通信，再使能电机。只有在机构人工调到真实水平位置时，才允许发送一次保存零点命令；正常上电应执行回零命令并轮询当前位置，确认连续两次落在零位容差内后才进入闭环。

每个控制周期先把  $\theta_d$  通过标定表换算成电机绝对角  $\phi_d$ ，再将角度换算为脉冲数，以绝对位置模式发送 `0xFD` 指令。当前换算采用 3200 脉冲/圈，电机速度初值 100 r/min、加速度参数 80。若通信超时、回零失败或当前位置越限，系统应立即停止发送闭环命令，执行停止/失能，并提示重新检查共地、串口和机械零点。

| 功能     | 当前协议/参数         | 安全要求                  |
|--------|-----------------|-----------------------|
| 通信检测   | 0x36 读取单圈位置     | 收到完整帧后才认为电机在线         |
| 保存水平零点 | 0x93，人工水平时仅执行一次 | 正常运行禁止每次上电重写零点        |
| 回到已存零点 | 0x9A，最长等待 10 s  | 角度容差 1.5°，连续 2 次满足才通过 |
| 绝对位置运动 | 0xFD，3200 脉冲/圈  | 先做球杆角度与电机角度双重限幅       |
| 停止/失能  | 0xFE / 0xF3     | 异常退出和回零失败必须执行         |

4.4 MaixCAM 控制流程

| 步骤    | 处理内容                                         |
|-------|----------------------------------------------|
| 初始化   | 校验参数 -> 打开 UART -> 读取电机位置 -> 使能 -> 回到已保存水平零点 |
| 接收球位  | 取得 x、valid、t_k；无效或过期数据不进入 PD                 |
| PD 计算 | 计算 e 和滤波微分，得到 $\theta_d$ 并执行角度限幅与斜率限制        |
| 机构映射  | 根据实测标定表将 $\theta_d$ 插值为电机绝对角 $\phi_d$        |
| 命令输出  | 角度变化超过死区时发送绝对位置命令                            |
| 故障处理  | 丢帧超时则回水平；串口/回零/越限故障则停止并失能                    |

5 联调顺序与待测参数

联调应按机械零位、步进电机方向、球杆角度标定、静态位置 PD、低速底盘循迹、动态滚球控制的顺序进行。每一步只改变一组参数，并保存球位、目标角、实际电机角、8 路红外状态、左右轮目标/实测速度和 PWM 日志。上一环节未通过限位与稳定性检查前，不进入下一环节。

| 待确认项目   | 需要记录的量               | 用于修正               |
|---------|----------------------|--------------------|
| 8 路红外安装 | 横向位置、离地高度、黑/白电平、环境光  | 位置权重、有效极性与脱线策略     |
| 底盘里程    | 实跑 1 m 的左右编码器计数      | 当前 3726 计数/m 的标定值  |
| 底盘速度    | 直线/弯道目标和实测速度、制动距离    | K1/K2 速度、减速里程与终点窗口 |
| 球杆机构    | L、r、连杆长度、零位相位、最大安全行程 | 运动学、转矩和软硬限位        |



|       |                                      |               |
|-------|--------------------------------------|---------------|
| 角度映射  | 至少 5 组 $\theta$ - $\phi$ 实测点，正负方向分别测 | 替换当前占位标定表     |
| 位置 PD | Kp、Kd、控制周期、D 项滤波、稳态误差                | 正式控制参数与到位判据   |
| 视觉接口  | 帧率、延迟、位置重复性、最长丢帧                     | PD 周期、超时与滤波参数 |

6 代码与文字一致性核对

**发现的待统一项：**这些差异不影响本阶段方案方向，但在提交报告和最终烧录前必须统一，否则报告参数会与实车程序不一致。

| 位置                      | 当前差异                                          | 建议                       |
|-------------------------|-----------------------------------------------|--------------------------|
| mid_line.h 与 mid_line.c | 注释写等间距-3.5 至+3.5；实际数组为-3、-2、-1、-0.1、0.1、1、2、3 | 以实测传感器位置确定一套权重，并同步代码和报告  |
| mid_line.c              | 注释多处写 30 帧/150 ms；实际宏为 20 帧，即 100 ms          | 统一窗口说明并实测误触发率            |
| mid_key.h 与调度器          | 头文件宏写 50 ms；调度器实际每 4 个 5 ms 控制帧调用，即 20 ms     | 统一速度渐变周期，避免参数理解错误        |
| main.py 与本方案            | 现程序为位置外环+速度内环的串级 PI；本阶段确认方案为位置 PD 直接输出角度      | 由视觉/控制组决定后改成同一算法，再据实更新报告 |

7 阶段结论

本阶段已经形成与本队实物方向一致的两条控制链：底盘由 8 路红外位置 PD、IMU 偏航 PI 和双轮编码器速度 PID 完成循迹；滚球机构由 MaixCAM 球位输入、位置 PD、机构角度标定和 UART 步进电机绝对位置控制完成调节。机械结构采用固定铰点与摇臂连杆实现球杆单端升降。下一阶段的重点不是继续扩写视觉描述，而是测量机构尺寸和  $\theta$ - $\phi$  映射、统一代码中的权重与调度参数，并在静态球杆上完成位置 PD 调参后再进行整车动态联调。

附录 A 本阶段对应代码入口

| 功能         | 文件/符号                                                          |
|------------|----------------------------------------------------------------|
| 8 路红外与横线检测 | Code/Middle/mid_line.c : MID_IR_ReadRaw 、 MID_Line_CalcError 、 |



|           |                                                                  |
|-----------|------------------------------------------------------------------|
|           | MID_Line_CheckEmergency                                          |
| 底盘三级闭环    | Code/Middle/mid_chassis.c: MID_Chassis_PosStep、YawStep、SpeedStep |
| PID 实现与初值 | Code/Middle/mid_pid.c: MID_PID_Pos_Calc、Yaw_Calc、Speed_Calc      |
| 5 ms 任务调度 | Code/App/app_scheduler.c: s_scheduler_step                       |
| 发车、里程与停车  | Code/Middle/mid_key.c、mid_encoder.c                              |
| 步进电机串口    | main.py: MotorController、pipe_angle_to_motor_angle               |
| 待改位置 PD   | main.py: 当前 CascadeController 需按最终算法统一                           |